Missing `constexpr` in `std::optional` and `std::variant`

Document #:	P2231R1
Date:	2021-02-11
Project:	Programming Language C++
Audience:	LWG

Reply-to: Barry Revzin <barry.revzin@gmail.com>

1 Revision History

Since [P2231R0], added a section about feature-test macros for this change.

2 Introduction

Each new language standard has increased the kinds of operations that we can do during constant evaluation time. C++20 was no different. With the adoption of [P1330R0], C++20 added the ability to change the active member of a union inside constexpr (the paper specifically mentions `std::optional`). And with the adoption of [P0784R7], C++20 added the ability to do placement new inside constexpr (by way of `std::construct_at`).

But even though the language provided the tools to make `std::optional` and `std::variant` completely constexpr-able, there was no such update to the library. This paper seeks to remedy that omission by simply adding `constexpr` to all the relevant places.

2.1 Implementing `std::optional`

I updated libstdc++'s implementation of `std::optional` to add these `constexpr`s (and replace the placement `new` calls with calls to `std::construct_at`) as follows, which compiles with both gcc and clang (demo):

```
-13,6 +11,7
 #include <bits/exception_defines.h>
 #include <bits/functional_hash.h>
 #include <bits/enable_special_members.h>
+#include <bits/stl_construct.h>
 #if __cplusplus > 201703L
 # include <compare>
 #endif
-206,7 +205,7
         { }

         // User-provided destructor is needed when _Up has non-trivial dtor.
```

```
-        ~_Storage() { }
+        constexpr ~_Storage() { }

         _Empty_byte _M_empty;
         _Up _M_value;
-217,12 +216,12
     bool _M_engaged = false;

     template<typename... _Args>
-    void
+    constexpr void
     _M_construct(_Args&&... __args)
     noexcept(is_nothrow_constructible_v<_Stored_type, _Args...>)
     {
-        ::new ((void *) std::__addressof(this->_M_payload))
-        _Stored_type(std::forward<_Args>(__args)...);
+        std::construct_at(std::__addressof(this->_M_payload),
+            std::forward<_Args>(__args)...);
         this->_M_engaged = true;
     }

-371,7 +370,7
     _Optional_payload& operator=(_Optional_payload&&) = default;

     // Destructor needs to destroy the contained value:
-    ~_Optional_payload()
+    constexpr ~_Optional_payload()
     {
         this->_M_reset();
     }
-388,17 +387,16
     // The _M_construct operation has !_M_engaged as a precondition
     // while _M_destruct has _M_engaged as a precondition.
     template<typename... _Args>
-    void
+    constexpr void
     _M_construct(_Args&&... __args)
     noexcept(is_nothrow_constructible_v<_Stored_type, _Args...>)
     {
-        ::new
-        (std::__addressof(static_cast<_Dp*>(this)->_M_payload._M_payload))
-        _Stored_type(std::forward<_Args>(__args)...);
+        std::construct_at(std::__addressof(static_cast<_Dp*>(this)-
        >_M_payload._M_payload._M_value),
+            std::forward<_Args>(__args)...);
         static_cast<_Dp*>(this)->_M_payload._M_engaged = true;
     }

-    void
+    constexpr void
     _M_destruct() noexcept
     {
         static_cast<_Dp*>(this)->_M_payload._M_destroy();
-754,7 +752,7
         : _Base(std::in_place, __il, std::forward<_Args>(__args)...) { }
```

```
     // Assignment operators.
-    optional&
+    constexpr optional&
     operator=(nullopt_t) noexcept
     {
         this->_M_reset();
-762,6 +760,7
     }

     template<typename _Up = _Tp>
+    constexpr
     enable_if_t<__and_v<__not_self<_Up>,
                         __not_<__and_<is_scalar<_Tp>,
                                       is_same<_Tp, decay_t<_Up>>>>,
-780,6 +779,7
     }

     template<typename _Up>
+    constexpr
     enable_if_t<__and_v<__not_<is_same<_Tp, _Up>>,
                         is_constructible<_Tp, const _Up&>,
                         is_assignable<_Tp&, _Up>,
-801,6 +801,7
     }

     template<typename _Up>
+    constexpr
     enable_if_t<__and_v<__not_<is_same<_Tp, _Up>>,
                         is_constructible<_Tp, _Up>,
                         is_assignable<_Tp&, _Up>,
-823,6 +824,7
     }

     template<typename... _Args>
+    constexpr
     enable_if_t<is_constructible_v<_Tp, _Args&&...>, _Tp&>
     emplace(_Args&&... __args)
     {
-832,6 +834,7
     }

     template<typename _Up, typename... _Args>
+    constexpr
     enable_if_t<is_constructible_v<_Tp, initializer_list<_Up>&,
                                    _Args&&...>, _Tp&>
     emplace(initializer_list<_Up> __il, _Args&&... __args)
-844,7 +847,7
     // Destructor is implicit, implemented in _Optional_base.

     // Swap.
-    void
+    constexpr void
     swap(optional& __other)
     noexcept(is_nothrow_move_constructible_v<_Tp>
              && is_nothrow_swappable_v<_Tp>)
```

```
-957,7 +960,7
         : static_cast<_Tp>(std::forward<_Up>(__u));
     }

-    void reset() noexcept
+    constexpr void reset() noexcept
     {
         this->_M_reset();
     }
-1187,7 +1190,7
 // _GLIBCXX_RESOLVE_LIB_DEFECTS
 // 2748. swappable traits for optionals
 template<typename _Tp>
-inline enable_if_t<is_move_constructible_v<_Tp> && is_swappable_v<_Tp>>
+constexpr inline enable_if_t<is_move_constructible_v<_Tp> && is_swappable_v<_Tp>>
 swap(optional<_Tp>& __lhs, optional<_Tp>& __rhs)
 noexcept(noexcept(__lhs.swap(__rhs))) {
     __lhs.swap(__rhs);
```

§ 2.2 Implementing `std::variant`

And likewise, here is a diff against libstdc++'s implementation of `std::variant` for the changes proposed in this paper, which also compiles on both gcc and clang (demo). This is slightly more complicated as we have to take more care with constructing and accessing the recursive union:

```
-121,7 +121,7
 __do_visit(_Visitor&& __visitor, _Variants&&... __variants);

 template <typename... _Types, typename _Tp>
-decltype(auto)
+constexpr decltype(auto)
 __variant_cast(_Tp&& __rhs) {
     if constexpr (is_lvalue_reference_v<_Tp>) {
         if constexpr (is_const_v<remove_reference_t<_Tp>>) {
-329,6 +329,11
         : _M_rest(in_place_index<_Np-1>, std::forward<_Args>(__args)...)
     { }

+    constexpr ~_Variadic_union() {}
+    constexpr ~_Variadic_union()
+        requires (std::is_trivially_destructible_v<_First> && ... &&
+                  std::is_trivially_destructible_v<_Rest>)  = default;
+
     _Uninitialized<_First> _M_first;
     _Variadic_union<_Rest...> _M_rest;
 };
-380,7 +385,7
           _M_index(_Np)
     { }

-    void _M_reset()
+    constexpr void _M_reset()
     {
         if (!_M_valid()) [[unlikely]]
```

```
             return;
-392,7 +397,7
         _M_index = variant_npos;
     }

-    ~_Variant_storage()
+    constexpr ~_Variant_storage()
     {
         _M_reset();
     }
-420,6 +425,10

 template<typename... _Types>
 struct _Variant_storage<true, _Types...> {
+    template<typename _Tp>
+    static constexpr size_t __index_of =
+        __detail::__variant::__index_of_v<_Tp, _Types...>;
+
     constexpr _Variant_storage() : _M_index(variant_npos) { }

     template<size_t _Np, typename... _Args>
-428,7 +437,7
           _M_index(_Np)
     { }

-    void _M_reset() noexcept
+    constexpr void _M_reset() noexcept
     {
         _M_index = variant_npos;
     }
-459,13 +468,16
     _Variant_storage<_Traits<_Types...>::_S_trivial_dtor, _Types...>;

 template<typename _Tp, typename _Up>
-void __variant_construct_single(_Tp&& __lhs, _Up&& __rhs_mem)
+constexpr void __variant_construct_single(_Tp&& __lhs, _Up&& __rhs_mem)
 {
-    void* __storage = std::addressof(__lhs._M_u);
     using _Type = remove_reference_t<decltype(__rhs_mem)>;
-    if constexpr (!is_same_v<_Type, __variant_cookie>)
-        ::new (__storage)
-        _Type(std::forward<decltype(__rhs_mem)>(__rhs_mem));
+    constexpr auto index = remove_reference_t<_Tp>::template __index_of<_Type>;
+
+    if constexpr (!is_same_v<_Type, __variant_cookie>) {
+        std::construct_at(std::addressof(__lhs._M_u),
+            in_place_index<index>,
+            _Type(std::forward<decltype(__rhs_mem)>(__rhs_mem)));
+    }
 }

 template<typename... _Types, typename _Tp, typename _Up>
-519,7 +531,7
     }
```

```
     template<typename _Up>
-    void _M_destructive_move(unsigned short __rhs_index, _Up&& __rhs)
+    constexpr void _M_destructive_move(unsigned short __rhs_index, _Up&& __rhs)
     {
         this->_M_reset();
         __variant_construct_single(*this, std::forward<_Up>(__rhs));
-527,7 +539,7
     }

     template<typename _Up>
-    void _M_destructive_copy(unsigned short __rhs_index, const _Up& __rhs)
+    constexpr void _M_destructive_copy(unsigned short __rhs_index, const _Up& __rhs)
     {
         this->_M_reset();
         __variant_construct_single(*this, __rhs);
-545,7 +557,7
     using _Base::_Base;

     template<typename _Up>
-    void _M_destructive_move(unsigned short __rhs_index, _Up&& __rhs)
+    constexpr void _M_destructive_move(unsigned short __rhs_index, _Up&& __rhs)
     {
         this->_M_reset();
         __variant_construct_single(*this, std::forward<_Up>(__rhs));
-553,7 +565,7
     }

     template<typename _Up>
-    void _M_destructive_copy(unsigned short __rhs_index, const _Up& __rhs)
+    constexpr void _M_destructive_copy(unsigned short __rhs_index, const _Up& __rhs)
     {
         this->_M_reset();
         __variant_construct_single(*this, __rhs);
-570,7 +582,7
     using _Base = _Move_ctor_alias<_Types...>;
     using _Base::_Base;

-    _Copy_assign_base&
+    constexpr _Copy_assign_base&
     operator=(const _Copy_assign_base& __rhs)
     noexcept(_Traits<_Types...>::_S_nothrow_copy_assign)
     {
-625,7 +637,7
     using _Base = _Copy_assign_alias<_Types...>;
     using _Base::_Base;

-    _Move_assign_base&
+    constexpr _Move_assign_base&
     operator=(_Move_assign_base&& __rhs)
     noexcept(_Traits<_Types...>::_S_nothrow_move_assign)
     {
-1033,12 +1045,10
 } // namespace __detail

 template<size_t _Np, typename _Variant, typename... _Args>
```

```
-void __variant_construct_by_index(_Variant& __v, _Args&&... __args) {
-    __v._M_index = _Np;
-    auto&& __storage = __detail::__variant::__get<_Np>(__v);
-    ::new ((void*)std::addressof(__storage))
-    remove_reference_t<decltype(__storage)>
-    (std::forward<_Args>(__args)...);
+constexpr void __variant_construct_by_index(_Variant& __v, _Args&&... __args) {
+    std::construct_at(std::addressof(__v),
+        in_place_index<_Np>,
+        std::forward<_Args>(__args)...);
 }

 template<typename _Tp, typename... _Types>
-1220,6 +1230,7
 constexpr decltype(auto) visit(_Visitor&&, _Variants&&...);

 template<typename... _Types>
+constexpr
 inline enable_if_t<(is_move_constructible_v<_Types> && ...)
                    && (is_swappable_v<_Types> && ...)>
 swap(variant<_Types...>& __lhs, variant<_Types...>& __rhs)
-1281,9 +1292,9
 {
 private:
     template <typename... _UTypes, typename _Tp>
-    friend decltype(auto) __variant_cast(_Tp&&);
+    friend constexpr decltype(auto) __variant_cast(_Tp&&);
     template<size_t _Np, typename _Variant, typename... _Args>
-    friend void __variant_construct_by_index(_Variant& __v,
+    friend constexpr void __variant_construct_by_index(_Variant& __v,
             _Args&&... __args);

     static_assert(sizeof...(_Types) > 0,
-1397,6 +1408,7
     { }

     template<typename _Tp>
+    constexpr
     enable_if_t<__exactly_once<__accepted_type<_Tp&&>>
                 && is_constructible_v<__accepted_type<_Tp&&>, _Tp>
                 && is_assignable_v<__accepted_type<_Tp&&>&, _Tp>,
-1421,6 +1433,7
     }

     template<typename _Tp, typename... _Args>
+    constexpr
     enable_if_t<is_constructible_v<_Tp, _Args...> && __exactly_once<_Tp>,
                 _Tp&>
     emplace(_Args&&... __args)
-1430,6 +1443,7
     }

     template<typename _Tp, typename _Up, typename... _Args>
+    constexpr
     enable_if_t<is_constructible_v<_Tp, initializer_list<_Up>&, _Args...>
```

```
                 && __exactly_once<_Tp>,
                 _Tp&>
-1440,6 +1454,7
     }

     template<size_t _Np, typename... _Args>
+    constexpr
     enable_if_t<is_constructible_v<variant_alternative_t<_Np, variant>,
                                    _Args...>,
                 variant_alternative_t<_Np, variant>&>
-1484,6 +1499,7
     }

     template<size_t _Np, typename _Up, typename... _Args>
+    constexpr
     enable_if_t<is_constructible_v<variant_alternative_t<_Np, variant>,
                                    initializer_list<_Up>&, _Args...>,
                 variant_alternative_t<_Np, variant>&>
-1540,7 +1556,7
         }
     }

-    void
+    constexpr void
     swap(variant& __rhs)
     noexcept((__is_nothrow_swappable<_Types>::value && ...)
              && is_nothrow_move_constructible_v<variant>)
```

§ 3 Wording

[Editor's note: The wording here just shows the added `constexpr`s in the synopses. They all need to be repeated in the specific wording for each function.]

Add `constexpr` to the `swap` in 20.6.2 [optional.syn] (and in 20.6.9 [optional.specalg]):

```
namespace std {
  // [optional.optional], class template optional
  template<class T>
    class optional;

  [...]

  // [optional.specalg], specialized algorithms
  template<class T>
-   void swap(optional<T>&, optional<T>&) noexcept(see below);
+   constexpr void swap(optional<T>&, optional<T>&) noexcept(see below);

  [...]

}
```

Add `constexpr` to the rest of the functions in 20.6.3.1 [optional.optional.general] (and likewise in 20.6.3.2 [optional.ctor], 20.6.3.3 [optional.dtor], 20.6.3.4 [optional.assign], 20.6.3.5 [optional.swap], and

20.6.3.7 [optional.mod]):

```
namespace std {
  template<class T>
  class optional {
  public:
    using value_type = T;

    // [optional.ctor], constructors
    constexpr optional() noexcept;
    constexpr optional(nullopt_t) noexcept;
    constexpr optional(const optional&);
    constexpr optional(optional&&) noexcept(see below);
    template<class... Args>
      constexpr explicit optional(in_place_t, Args&&...);
    template<class U, class... Args>
      constexpr explicit optional(in_place_t, initializer_list<U>, Args&&...);
    template<class U = T>
      constexpr explicit(see below) optional(U&&);
    template<class U>
-     explicit(see below) optional(const optional<U>&);
+     constexpr explicit(see below) optional(const optional<U>&);
    template<class U>
-     explicit(see below) optional(optional<U>&&);
+     constexpr explicit(see below) optional(optional<U>&&);

    // [optional.dtor], destructor
-   ~optional();
+   constexpr ~optional();

    // [optional.assign], assignment
-   optional& operator=(nullopt_t) noexcept;
+   constexpr optional& operator=(nullopt_t) noexcept;
    constexpr optional& operator=(const optional&);
    constexpr optional& operator=(optional&&) noexcept(see below);
-   template<class U = T> optional& operator=(U&&);
-   template<class U> optional& operator=(const optional<U>&);
-   template<class U> optional& operator=(optional<U>&&);
-   template<class... Args> T& emplace(Args&&...);
-   template<class U, class... Args> T& emplace(initializer_list<U>, Args&&...);
+   template<class U = T> constexpr optional& operator=(U&&);
+   template<class U> constexpr optional& operator=(const optional<U>&);
+   template<class U> constexpr optional& operator=(optional<U>&&);
+   template<class... Args> constexpr T& emplace(Args&&...);
+   template<class U, class... Args> constexpr T& emplace(initializer_list<U>,
        Args&&...);

    // [optional.swap], swap
-   void swap(optional&) noexcept(see below);
+   constexpr void swap(optional&) noexcept(see below);

    // [optional.observe], observers
    constexpr const T* operator->() const;
    constexpr T* operator->();
    constexpr const T& operator*() const&;
```

```
    constexpr T& operator*() &;
    constexpr T&& operator*() &&;
    constexpr const T&& operator*() const&&;
    constexpr explicit operator bool() const noexcept;
    constexpr bool has_value() const noexcept;
    constexpr const T& value() const&;
    constexpr T& value() &;
    constexpr T&& value() &&;
    constexpr const T&& value() const&&;
    template<class U> constexpr T value_or(U&&) const&;
    template<class U> constexpr T value_or(U&&) &&;

    // [optional.mod], modifiers
-   void reset() noexcept;
+   constexpr void reset() noexcept;

  private:
    T *val;         // exposition only
  };

  template<class T>
    optional(T) -> optional<T>;
}
```

Add `constexpr` to `swap` in 20.7.2 [variant.syn] (and likewise in 20.7.10 [variant.specalg]):

```
namespace std {
  // [variant.variant], class template variant
  template<class... Types>
    class variant;

  [...]

  // [variant.specalg], specialized algorithms
  template<class... Types>
-   void swap(variant<Types...>&, variant<Types...>&) noexcept(see below);
+   constexpr void swap(variant<Types...>&, variant<Types...>&) noexcept(see below);

  [...]
}
```

Add `constexpr` to the rest of the functions in 20.7.3.1 [variant.variant.general] (and likewise in 20.7.3.3 [variant.dtor], 20.7.3.4 [variant.assign], 20.7.3.5 [variant.mod], and 20.7.3.7 [variant.swap]):

```
namespace std {
  template<class... Types>
  class variant {
  public:
    // [variant.ctor], constructors
    constexpr variant() noexcept(see below);
    constexpr variant(const variant&);
    constexpr variant(variant&&) noexcept(see below);

    template<class T>
      constexpr variant(T&&) noexcept(see below);
```

```
    template<class T, class... Args>
      constexpr explicit variant(in_place_type_t<T>, Args&&...);
    template<class T, class U, class... Args>
      constexpr explicit variant(in_place_type_t<T>, initializer_list<U>, Args&&...);

    template<size_t I, class... Args>
      constexpr explicit variant(in_place_index_t<I>, Args&&...);
    template<size_t I, class U, class... Args>
      constexpr explicit variant(in_place_index_t<I>, initializer_list<U>, Args&&...);

    // [variant.dtor], destructor
-   ~variant();
+   constexpr ~variant();

    // [variant.assign], assignment
    constexpr variant& operator=(const variant&);
    constexpr variant& operator=(variant&&) noexcept(see below);

-   template<class T> variant& operator=(T&&) noexcept(see below);
+   template<class T> constexpr variant& operator=(T&&) noexcept(see below);

    // [variant.mod], modifiers
-   template<class T, class... Args>
-     T& emplace(Args&&...);
-   template<class T, class U, class... Args>
-     T& emplace(initializer_list<U>, Args&&...);
-   template<size_t I, class... Args>
-     variant_alternative_t<I, variant<Types...>>& emplace(Args&&...);
-   template<size_t I, class U, class... Args>
-     variant_alternative_t<I, variant<Types...>>& emplace(initializer_list<U>,
        Args&&...);
+   template<class T, class... Args>
+     constexpr T& emplace(Args&&...);
+   template<class T, class U, class... Args>
+     constexpr T& emplace(initializer_list<U>, Args&&...);
+   template<size_t I, class... Args>
+     constexpr variant_alternative_t<I, variant<Types...>>& emplace(Args&&...);
+   template<size_t I, class U, class... Args>
+     constexpr variant_alternative_t<I, variant<Types...>>&
        emplace(initializer_list<U>, Args&&...);

    // [variant.status], value status
    constexpr bool valueless_by_exception() const noexcept;
    constexpr size_t index() const noexcept;

    // [variant.swap], swap
-   void swap(variant&) noexcept(see below);
+   constexpr void swap(variant&) noexcept(see below);
  };
}
```

§ 3.1 Feature-test macro

The usual policy for constexprification is that we bump the `__cpp_lib_constexpr_`*`HEADER`* macro. In this case, we do not have such a macro for either `optional` or `variant`. Given that this paper finishes marking the entirety of the API as `constexpr`, it also does not make sense to add a new constexpr macro solely for this change.

We do have `__cpp_lib_optional` and `__cpp_lib_variant`, and this does seem like a necessary API change to be reflected in a feature test, so we could instead bump those macros.

Change 17.3.2 [version.syn]:

```
- #define __cpp_lib_optional      201606L // also in <optional>
- #define __cpp_lib_variant       201606L // also in <variant>
+ #define __cpp_lib_optional      2021XXL // also in <optional>
+ #define __cpp_lib_variant       2021XXL // also in <variant>
```

4 Acknowledgements

Thanks to Tim Song for all the help. Thanks to Jonathan Wakely for looking over the paper, pointing out how repetitive the introduction was, and pointing out how repetitive the introduction was.

5 References

[P0784R7] Daveed Vandevoorde, Peter Dimov,Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More constexpr containers. https://wg21.link/p0784r7

[P1330R0] Louis Dionne, David Vandevoorde. 2018-11-10. Changing the active member of a union inside constexpr. https://wg21.link/p1330r0

[P2231R0] Barry Revzin. 2020-10-14. Add further constexpr support for optional/variant. https://wg21.link/p2231r0